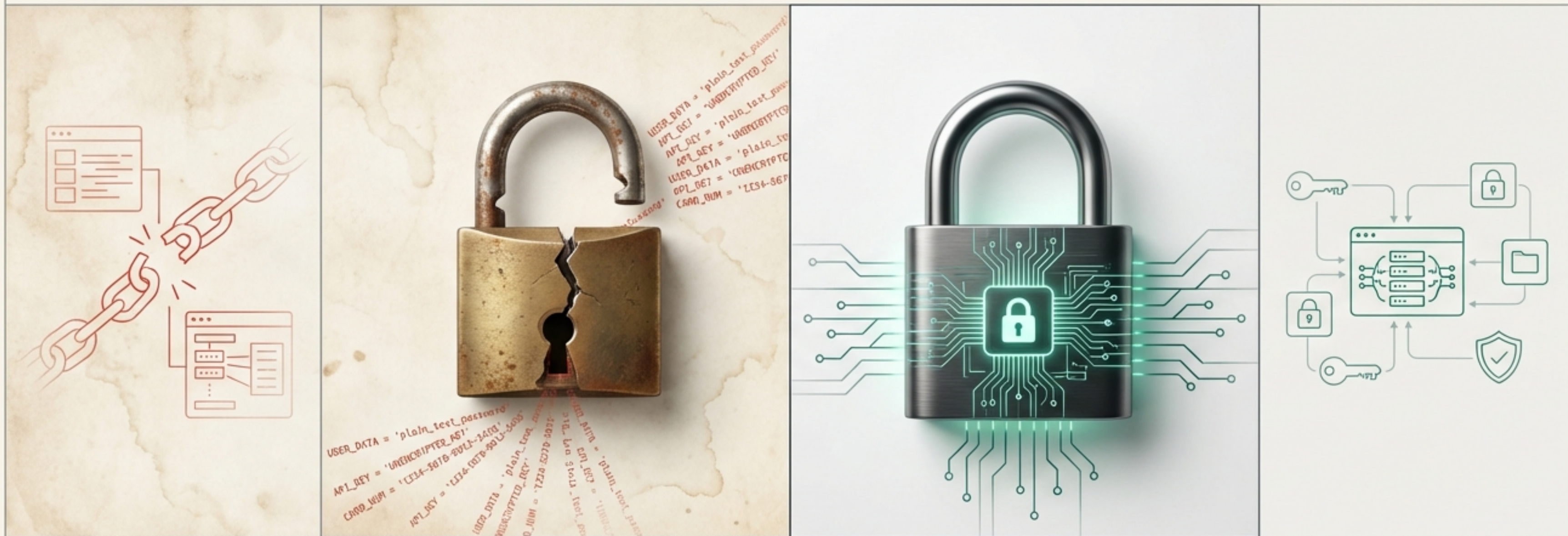


پروژه امنیت محاسبات: داستان دو سیستم



تحلیلی بر آسیب‌پذیری‌ها و راهکارهای امن در ذخیره‌سازی و انتقال داده

چرا این پروژه اهمیت دارد؟

مسئله اصلی: داده‌های حساس در خطر

روزانه، اطلاعات حساس کاربران (مانند رمزهای عبور و اطلاعات کارت اعتباری) به دلیل پیاده‌سازی‌های ناامن در معرض خطر قرار می‌گیرند.



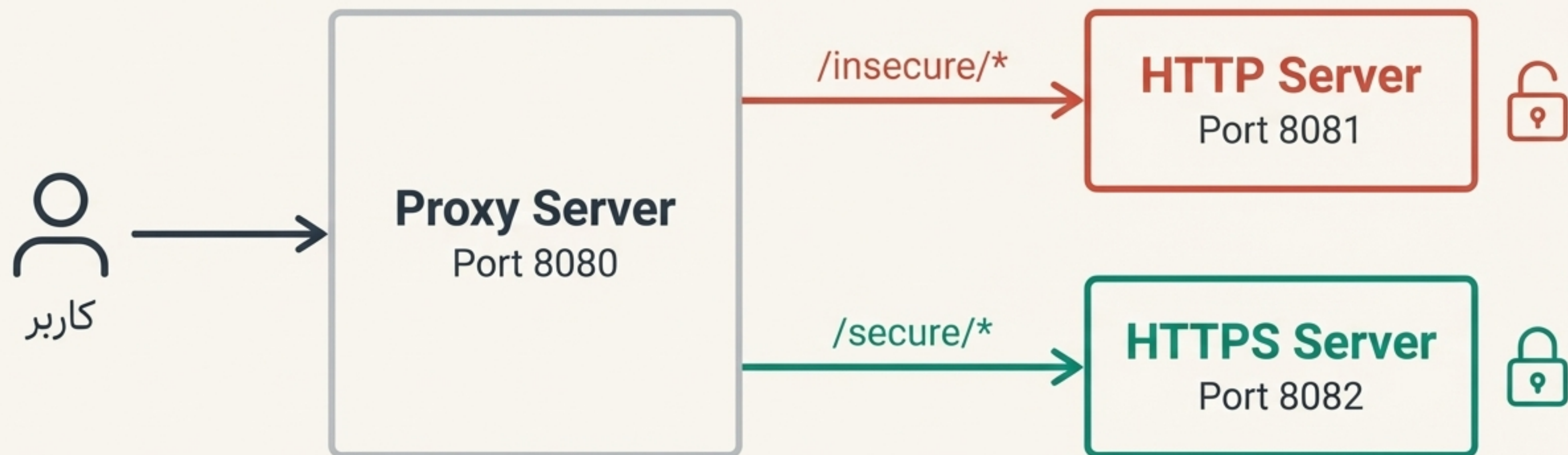
این پروژه به صورت عملی نشان می‌دهد که:

- چگونه **اشتباهات رایج امنیتی**، از جمله ذخیره‌سازی **متن** ساده و **انتقال ناامن**، رخ می‌دهند.

- چگونه می‌توان با استفاده از **روش‌های استاندارد** و **صحیح**، از این داده‌ها به طور **مؤثر محافظت** کرد.

در ادامه، ما سفر یک کاربر را در دو مسیر موازی دنبال خواهیم کرد: **مسیر ناامن** و **مسیر امن**.

معماری پروژه: جداسازی هوشمندانه ترافیک با Reverse Proxy

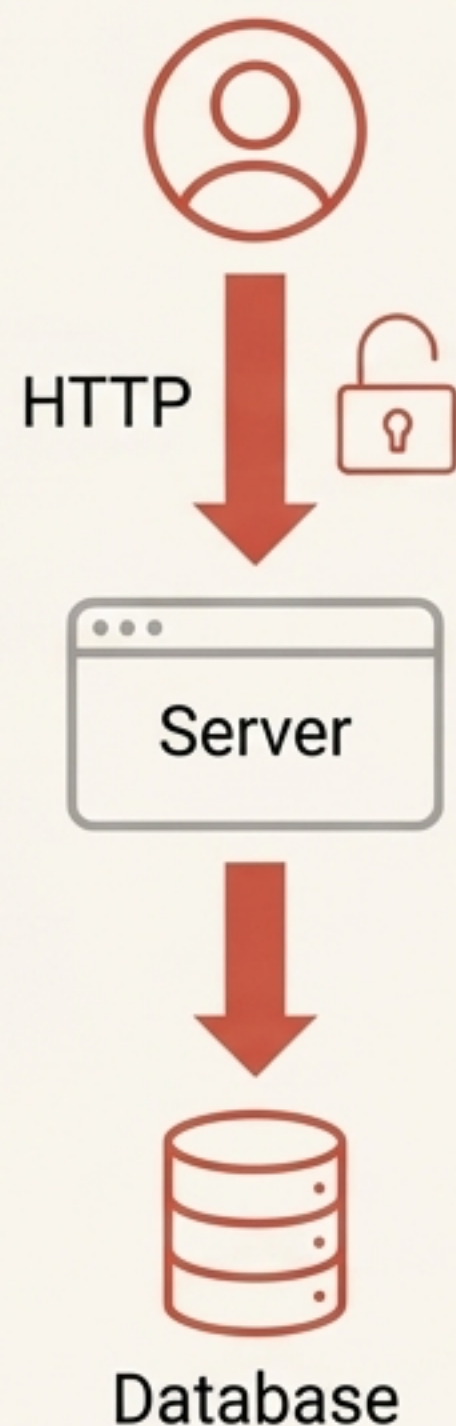


نکته کلیدی:

این معماری، که مشابه محیط‌های واقعی (Production) است، امنیت را از اولین نقطه تماس اعمال می‌کند. با هدایت هوشمندانه ترافیک، اطمینان حاصل می‌شود که مسیرهای امن هرگز از طریق یک کانال ناامن پردازش نمی‌شوند.

سفر اول، ثبت نام: مسیر ناامن و اولین اشتباه

سناریو: یک کاربر جدید در مسیر `insecure/register` ثبت نام می کند.



هشدار: کد ناامن

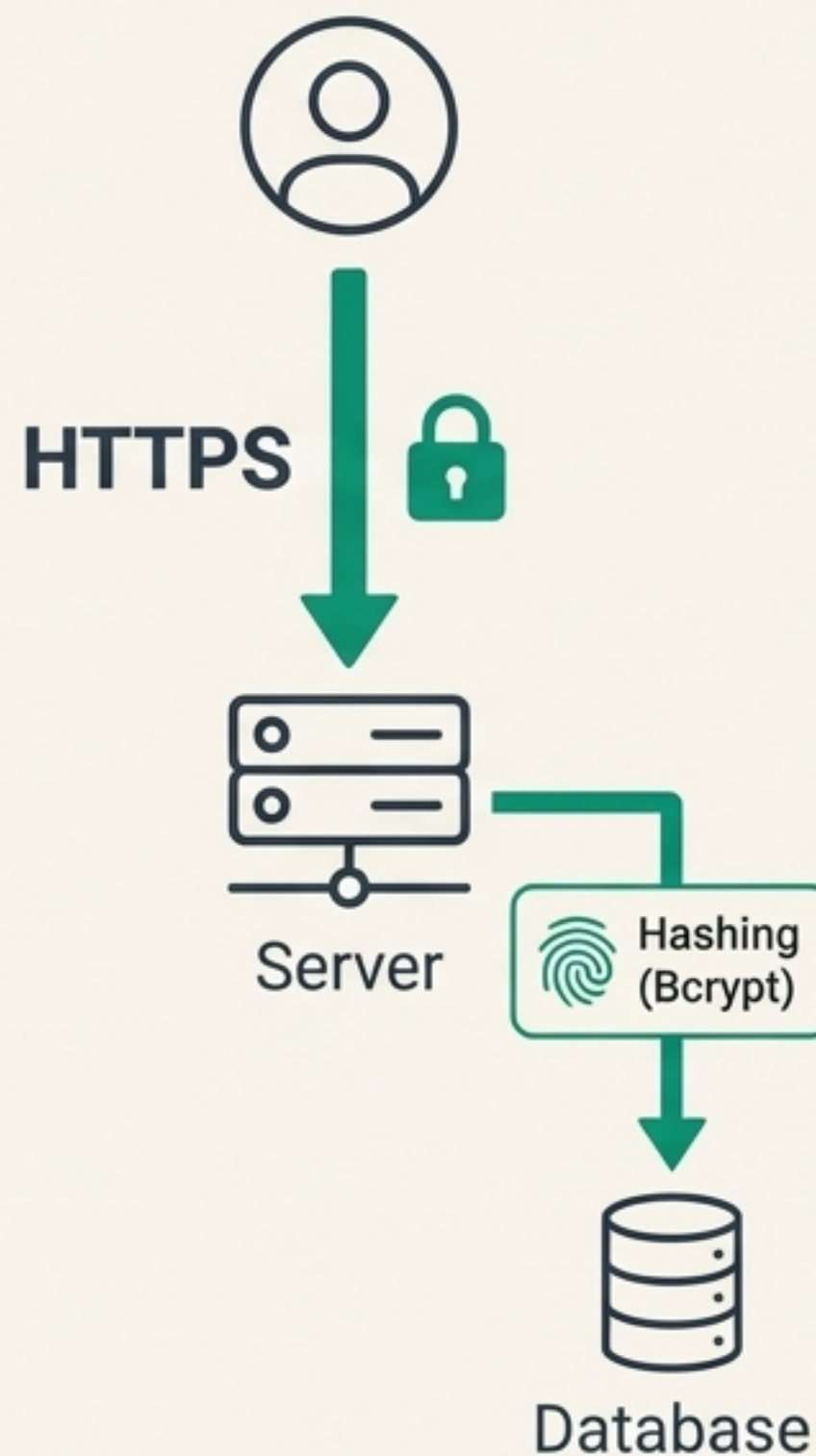
رمز عبور مستقیماً از فرم گرفته شده و ذخیره می شود

```
password = request.form['password']
new_user = User(username=username, password=password)
db.session.add(new_user)
db.session.commit()
```

نتیجه: رمز عبور کاربر دقیقاً همانطور که وارد شده، در پایگاه داده ذخیره می شود. یک نشت اطلاعاتی کوچک می تواند کل سیستم و اعتبار کاربران را به خطر اندازد.

سفر اول، ثبت نام: مسیر امن و راه حل صحیح

سناریو: یک کاربر جدید در مسیر `/secure/register` ثبت نام می کند.



پیاده سازی امن

رمز عبور قبل از ذخیره سازی با Bcrypt هش می شود

```
password = request.form['password']
hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
new_user = User(username=username, password=hashed_password)
db.session.add(new_user)
db.session.commit()
```

نتیجه: فقط نسخه هش شده و غیرقابل بازگشت رمز عبور در پایگاه داده ذخیره می شود. رمز عبور اصلی هرگز در سیستم باقی نمی ماند و حتی در صورت نشت، داده ها امن هستند.

لحظه حقیقت: مقایسه محتوای دو پایگاه داده

پایگاه داده ناامن (Insecure DB) 

Field	Value
id	1
username	testuser
password	MySecretPa\$\$w0rd

پایگاه داده امن (Secure DB) 

Field	Value
id	1
username	testuser
password	\$2b\$12\$aBcDeFgHiJkLmNoPqRsTu.xYz...

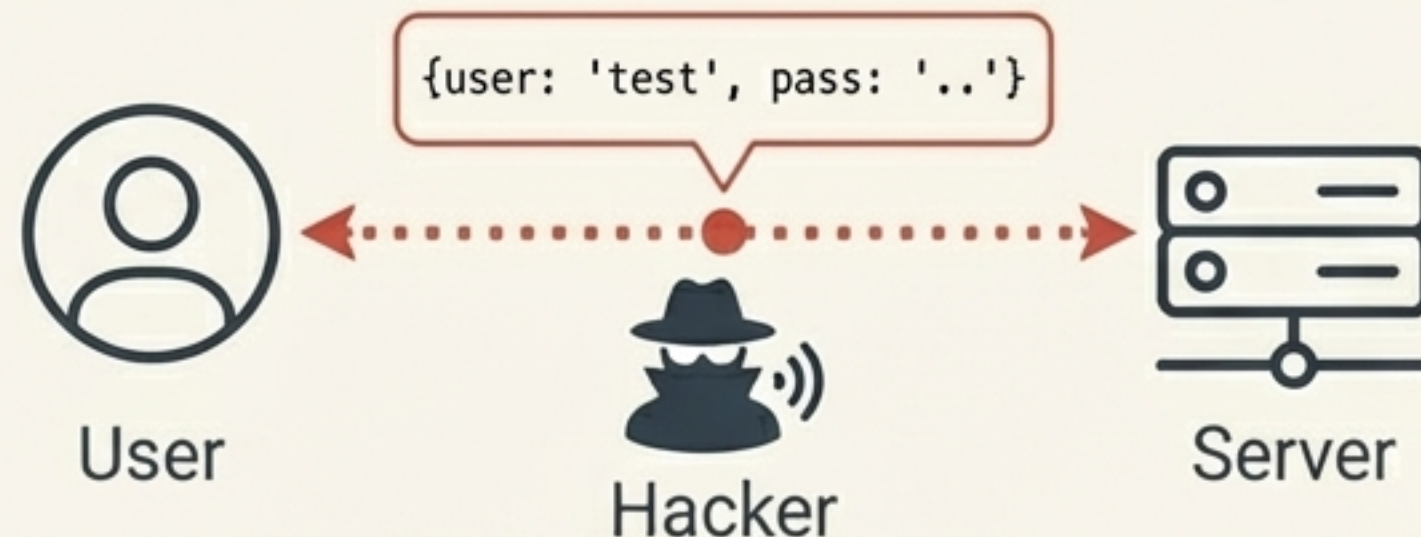
نتیجه‌گیری کلیدی: در سمت چپ، یک فاجعه امنیتی در انتظار وقوع است. در سمت راست، داده‌ها حتی در صورت نشت اطلاعات پایگاه داده نیز محافظت می‌شوند.

سفر دوم، ورود: مسیر ناامن و دو خطر بزرگ

سناریو: کاربر در مسیر `insecure/login` تلاش می‌کند وارد شود.

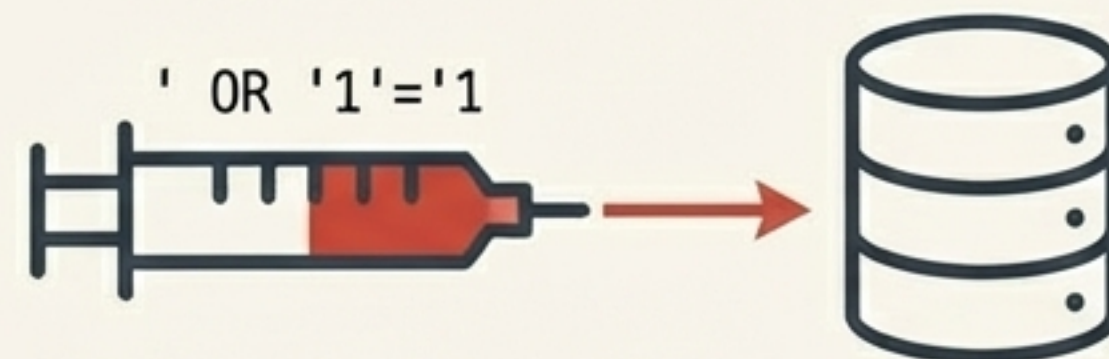
آسیب‌پذیری ۱: انتقال ناامن (Insecure Transmission)

تمام اطلاعات ورود (نام کاربری و رمز عبور) از طریق HTTP به صورت متن ساده ارسال می‌شوند و توسط هر کسی در شبکه قابل شنود (Sniffing) هستند.



آسیب‌پذیری ۲: تزریق SQL (SQL Injection)

کوئری‌های SQL با الحاق مستقیم رشته‌ها ساخته می‌شوند. این کار مهاجم اجازه می‌دهد تا با ورودی‌های دستکاری‌شده، کوئری پایگاه داده را تغییر داده و به داده‌های غیرمجاز دسترسی پیدا کند.



سفر دوم، ورود: مسیر امن و دفاع چندلایه

سناریو: کاربر در مسیر `/secure/login` وارد می‌شود.

راه حل ۱: ارتباطات رمزگذاری شده (Enforced HTTPS)

یک Middleware تضمین می‌کند که تمام ارتباطات با مسیرهای `/secure/*` به صورت خودکار به HTTPS هدایت و رمزگذاری می‌شوند. این کار از شنود اطلاعات در حال انتقال جلوگیری می‌کند.



راه حل ۲: کوئری‌های پارامتری (Parameterized Queries)

استفاده از ORM (مانند SQLAlchemy) یا کوئری‌های پارامتری، به طور کامل از حملات تزریق SQL جلوگیری می‌کند. در این روش، ورودی کاربر هرگز به عنوان کد اجرایی تفسیر نمی‌شود، بلکه همیشه به عنوان داده در نظر گرفته می‌شود.



فراتر از اصول اولیه: سایر آسیب پذیری ها



آسیب پذیری: رمزگذاری ضعیف (Weak Encryption)

مثال: استفاده از Base64 که یک روش کدگذاری (Encoding) است، نه رمزنگاری (Encryption).
راه حل: استفاده از الگوریتم های رمزنگاری استاندارد و معتبر.



آسیب پذیری: افشای داده های حساس در API

مثال: بازگرداندن کل آبجکت کاربر، شامل هش رمز عبور، در پاسخ های API.
راه حل: فیلتر کردن دقیق پاسخ های API و بازگرداندن تنها اطلاعات ضروری.



آسیب پذیری: عدم اعتبارسنجی ورودی (No Input Validation)

مثال: پذیرش هر نوع ورودی بدون بررسی فرمت، طول یا محتوا.
راه حل: پاک سازی (Sanitization) و اعتبارسنجی (Validation) تمام ورودی های کاربر.

خلاصه نبرد: ناامن در برابر امن

آسیب‌پذیری (Vulnerability) ●	راه‌حل امن (Secure Solution) ●
ذخیره رمز عبور به صورت متن ساده	هش کردن با الگوریتم Bcrypt
انتقال داده با HTTP	اجبار به استفاده از HTTPS
تزریق SQL (SQL Injection)	کوئری‌های پارامتری (استفاده از ORM)
رمزگذاری ضعیف (مانند Base64)	الگوریتم‌های رمزنگاری استاندارد
افشای داده‌های حساس در API	فیلتر کردن دقیق پاسخ‌های API
عدم اعتبارسنجی ورودی کاربر	پاک‌سازی و اعتبارسنجی ورودی

خودتان تجربه کنید: اجرای معماری Reverse Proxy

۱. نصب وابستگی‌ها:

```
>_ pip install -r requirements.txt
```

۲. ایجاد گواهی SSL (برای HTTPS):

```
>_ ./generate_cert.sh
```

۳. اجرای معماری کامل:

```
>_ python run_proxy.py
```

نقطه دسترسی اصلی:

`http://localhost:8080`

فراخوان به اقدام: با اجرای پروژه، می‌توانید این مفاهیم را به صورت زنده مشاهده کرده، ترافیک شبکه را بررسی کنید و تفاوت‌ها را در پایگاه داده مشاهده نمایید.

امنیت یک انتخاب است، نه یک گزینه



این پروژه به وضوح نشان می‌دهد که پیاده‌سازی روش‌های امن، پیچیدگی زیادی به کد اضافه نمی‌کند، اما تفاوت بین یک سیستم ایمن و یک سیستم آسیب‌پذیر را رقم می‌زند.



****هشدار نهایی (مهم):****

کد ناامن موجود در این پروژه فقط و فقط برای اهداف آموزشی است. ****هرگز**** از الگوهای ناامن نمایش داده شده در یک محیط واقعی (Production) استفاده نکنید.

اطلاعات پروژه و منابع بیشتر



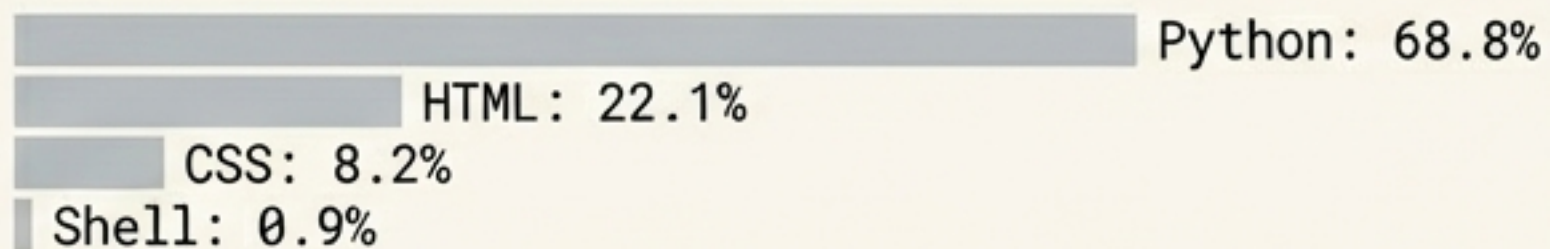
اسکن کنید تا به مخزن پروژه دسترسی پیدا کنید

مشارکت‌کنندگان (Contributors):

AmirAli-Abedi

MahdiOTET

زبان‌های استفاده شده در پروژه:



لینک مخزن GitHub:

github.com/AmirAli-Abedi/FCS-project